Technical report — Sturmglocke

A sovereign crisis-staff training game where every alert, caller and debrief is Apertus 1.5 — and every alert is checked before it goes out.

- **Track:** Track 2B — Sturmglocke
- **Event:** Online
- **Team:** Team Barry — Tushar Agarwal
- **Demo video:** https://youtu.be/L0UWGKaWAWQ · **Code:** https://github.com/TusharTechs/sturmglocke
- **Run:** `make run` (hosted Apertus) · `make airgap` / `make airgap-mac` (local Apertus 8B, no network)

1. Summary

Problem. On 28 May 2025 a glacier collapse buried 90 % of Blatten (VS); roughly 300 people had been evacuated in time because the cantonal geologists and the regional crisis staff acted early. Each of Switzerland's 2,110 municipalities relies on such a crisis staff (Gemeindeführungsorgan / regionales Führungsorgan) that must train regularly, decide under time pressure and warn residents, workers and tourists in several languages. Two things make language models attractive and dangerous here: multilingual alerts must be written in minutes, and during a disaster the network may be gone.

Approach. *Sturmglocke* ("storm bell", the old village alarm bell) is a pixel-art training game for crisis staffs. A fictional alpine valley, 146 residents in five languages, a debris flow building above two hamlets. Apertus 1.5 voices every caller, the cantonal geologist, the chief of staff and the after-action review, and it writes every alert. The core component is an **alert fidelity guard**: alerts are composed from structured facts, rendered per language with the **official Swiss hazard term locked** (TERMDAT / Pledari Grond), read back by the model, checked deterministically and repaired once before the commander approves. Residents on the map act on the version *they* can read, so a translation error becomes visible. Everything runs on Swiss infrastructure (CSCS) or fully air-gapped on a laptop.

Headline results (720 alert texts, 30 cases × FR/IT/EN/RM × 3 conditions × 2 models; details §5):

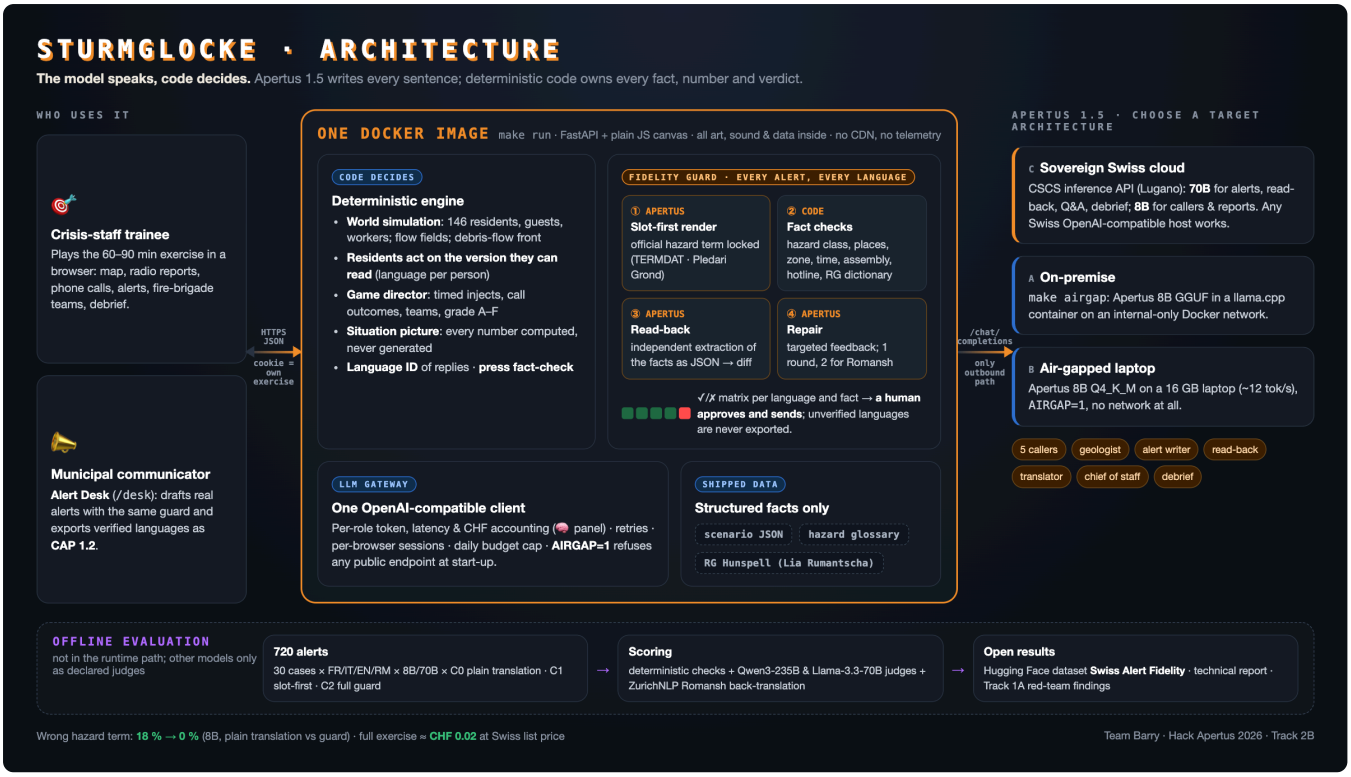
	Apertus 8B	**Apertus 70B**
Plain "translate the German alert": **wrong hazard process** named	**18 %**	5 %
Plain translation: official term + all facts preserved	45 %	71 %
Sturmglocke guard: official term + all facts preserved	**100 %** [97–100]	**100 %** [97–100]
Guard: meaning preserved FR/IT/EN (independent Qwen3 judge)	99 %	98 %
Chief of staff asserts facts not in the situation picture (generic prompt → ours)	20/28 → **0/28**	19/28 → **0/28**

The guarded 8B runs **offline on a 16 GB M1 laptop** (≈12 tok/s; full 4-language guarded alert in 65 s). A complete exercise measured **51 Apertus calls, 19.7k tokens ≈ CHF 0.02** at Swiss list price. The same guard also ships as the **Alert Desk** (`/desk`) for real communicators, exporting verified languages as **CAP 1.2** XML.

How the judging criteria are met

Criterion	**Evidence**
Purposeful use of AI	Apertus does what needs language — alerts in 5 languages, read-back, callers, terminology-locked translation and subtitles, debrief — while code decides outcomes; a live 🧠 panel shows every call by role (§3)
Technical rigour	720-alert benchmark with model-free primary metric, independent open-weights judge, Romansh back-translation + official dictionary, honesty eval, 14 offline tests, every number regenerable (§5)
Value, cost & scalability	Training for 2,110 municipalities; CHF 0.02 per exercise or free on owned hardware; scenarios are JSON; the guard transfers to real alerting via Alert Desk + CAP (§5.5, `docs/SCENARIOS.md`)
Sovereign deployability	Swiss cloud (CSCS), on-prem compose with internal-only model network, air-gapped laptop verified; `AIRGAP=1` refuses public endpoints; no external assets (tested) (§2)
Implementation feasibility	`make run` on a clean checkout, Docker, deployment & pilot guide (`docs/DEPLOYMENT.md`), graceful degradation when the model is down, mobile layout

2. Architecture



Modules (`src/sturmglocke/`): `game.py` clock, injects, player actions, Lagebild, press check · `world.py` deterministic valley and residents · `alerts.py` the fidelity guard · `agents.py` every Apertus role · `cap.py` CAP 1.2 export · `llm.py` the only outbound path (OpenAI-compatible `LLM_BASE_URL`; `AIRGAP=1` refuses public hosts) · `main.py` FastAPI, one exercise per browser session.

Design principle: the model speaks, code decides. Who is safe, when the hazard hits, every number in the situation picture, whether a phone instruction reached a group and whether an alert passes are decided by deterministic, seeded code. Apertus produces language: callers' lines, reports, alert texts, the read-back, staff answers, the debrief.

Alert pipeline. The commander fills structured slots (hazard, zone, hamlets, action, leave-by time, assembly point, hotline). For each language Apertus writes the alert from the slots (not by translating free text), with the official hazard term, the zone word and names given verbatim. Then: (1) deterministic checks — official term present and no term that official sources assign to a *different* process, names verbatim, zone, time in any local format, assembly point, hotline, and for Romansh every word against the official RG dictionary; (2) an independent read-back — Apertus reads only the text and reports hazard type, action and whether the time is a deadline; (3) on any failure, one targeted repair round (two for Romansh); (4) the per-slot verdict is shown to the commander (✓/✗ chips) and drives the simulation.

Target architecture (mandatory)

Deployable in **all three** architectures; the only runtime dependency is an OpenAI-compatible Apertus endpoint.

- **a) On-premise** — `make airgap`: app + `llama.cpp` serving Apertus 1.5 8B (GGUF) in Docker; the model container sits on an `internal: true` network with no route to the internet.
- **b) Air-gapped** — verified on an M1 MacBook (16 GB) with Wi-Fi off: `make llm-mac` (Metal) + `make airgap-mac`. With `AIRGAP=1` the app resolves `LLM_BASE_URL` at start-up and **refuses to run** unless it is a private/loopback address (tested: CSCS rejected, local accepted). Frontend: no CDN, fonts, images or audio files — art and sirens are generated in code.
- **c) Sovereign Swiss cloud** — default `make run` against the CSCS inference service (Swiss National Supercomputing Centre, Lugano); equally any Swiss provider serving Apertus via an OpenAI-compatible API (e.g. Phoeniqs, Infomaniak, Swisscom). `docker-compose.cloud.yml` runs the same image on a Swiss VM behind HTTPS (Caddy), with one exercise per browser and a daily call cap that protects the API budget.

Build time vs runtime. Build: Python wheels, the `python:3.12-slim` image, the GGUF file (≈5.1 GB, downloaded once). Runtime: nothing but the LLM endpoint. No telemetry, no analytics.

3. Use of Apertus

- **Models:** `swiss-ai/Apertus-v1.5-70B` (alerts, read-back, chief of staff, debrief) and `swiss-ai/Apertus-v1.5-8B` (callers, geologist, translation aid) via CSCS; offline: Apertus 1.5 8B text Q4_K_M GGUF (`Colby/apertus-v1.5-8b-`

`text-Q4_K_M-GGUF`, converted from the FHNW text checkpoint `andreasmartin/apertus-v1.5-8b-text`) on `llama.cpp` 0.5.0 with the embedded template.

- **How it is used:** inference with role prompts; JSON mode for read-back and order extraction (one repair turn, lenient parser); thinking only via the separate `-thinking` model and never combined with tool calls (documented 1.5 limitation). Temperature 0 for alerts and checks, 0.3–0.6 for characters.
- **Probing first.** Before building we probed the endpoint (`docs/probes/FINDINGS.md`): latency 1–2.5 s, 4/4 JSON, working tool calls — and the key finding that drove the design: translating a German debris-flow alert, *every* structural fact survived but the hazard became *glissement de terrain*, *frana*, *fango* or *landslide*, and a fictional valley became the real *Silvaplana*.

AI use disclosure

The product runs on Apertus only. Every model call Sturmglocke makes at runtime (callers, geologist, alert writing, read-back, repair, translation, subtitles, chief of staff, debrief, Alert Desk) goes to Apertus 1.5 through `llm.py`, the only outbound path. No other model is called, bundled or required by the submission, so the deployed product is 100 % Apertus. Other AI was used only around the product, never in it:

Tool	Role	In the product?
Qwen3-235B-A22B-Instruct-2507, Llama-3.3-70B-Instruct (open weights, Hugging Face Inference)	independent judges in the offline evaluation (§5)	no
ZurichNLP/romansh-nllb-1.3b (open weights, CC-BY-NC)	back-translates Romansh alerts for evaluation (§5)	no
Claude Code (Anthropic), AI coding assistant	helped write code, tests and documentation under the author's direction, as the rules permit	no
Apple macOS text-to-speech (local)	narration voice of the demo video (Apertus has no speech output)	no

All alert texts, character lines and model responses in the repository and dataset were generated by Apertus 1.5; the judges' verdicts in the dataset are labelled with the judge model's name.

4. Data

All data is either authored structure or official reference material; no personal data.

Data	Source	Licence
Scenario `data/scenarios/silvatal.json`	authored structured facts; fictional valley and people	CC-BY-4.0
Hazard glossary `data/glossary/hazards.json`	TERMDAT entries 216219, 216229, 64271, 215892, 502116, 216425 … (Federal Chancellery); Pledari Grond (verified via its API)	single terms with source ids
Romansh dictionary `data/hunspell/`	Lia Rumantscha / Pro Svizra Rumantscha RG Hunspell 2.2.2	MIT
Siren behaviour (red-team companion)	BABS / Alertswiss pages in DE/FR/IT/EN	facts, cited
Evaluation cases + 720 scored alerts	seeded generator, invented names; Hugging Face dataset `TechTush/swiss-alert-fidelity`	CDLA-Permissive-2.0

Swiss official texts (laws, decisions) are not copyright-protected (Art. 5 URG); we nonetheless reproduce only single terms.

5. Evaluation

All runs are reproducible from `src/sturmglocke/evals` (`make eval`); every number below is regenerated by `python -m sturmglocke.evals.report_numbers` into `docs/eval/REPORT_NUMBERS.md`.

5.1 Alert fidelity. 30 seeded cases (5 hazards × 6 invented municipalities with German/French/Italian/Romansh-looking names) × FR/IT/EN/RM × 2 models × 3 conditions: **C0** Apertus writes the German alert, then plain-translates (common practice); **C1** slot-first rendering with the term locked, no checking; **C2** C1 + checks, read-back, repair (what the game ships). Primary scoring is deterministic and model-free; meaning is scored by an independent judge (Qwen3-235B) reading the text alone.

model	cond	official term	WRONG process	missing / non-word	official + all facts	meaning OK FR/IT/EN
8B	C0	48 %	**18 %**	18 %	45 % [36–54]	68 %

model	cond	official term	WRONG process	missing / non-word	official + all facts	meaning OK FR/IT/EN
8B	C2	100 %	0 %	0 %	**100 % [97–100]**	99 %
70B	C0	72 %	5 %	12 %	71 % [62–78]	90 %
70B	C2	100 %	0 %	0 %	**100 % [97–100]**	98 %

A second open-weights judge (Llama-3.3-70B) agrees with Qwen3 on 95.6 % of French/Italian/English verdicts (Cohen's κ = 0.72) but only 63.9 % on Romansh (κ = 0.22) — general models cannot judge Romansh, hence §5.2. All 720 texts with both verdicts are published as the dataset *Swiss Alert Fidelity* (`TechTush/swiss-alert-fidelity`, CDLA-Permissive-2.0).

Typical C0 errors: *Murgang* → *éboulement* (official term for rockslide), *glissement de terrain*, *coulée de boue*, *frana*, *fango*, *landslide*; *Lawine* → the non-words *Lavine/Lavina*; warnings turned into past events ("*Lavine a touché*"). Most of the gain comes from slot-first rendering with the locked term (C1: 98–99 %); the guard closes the rest and never passed a text that the deterministic score failed (0/240). It flagged 15–28 of 120 alerts per model to the commander and repaired 25–32.

5.2 Romansh. The judge cannot read Romansh: it labelled every official RG hazard term (*crudada da crappa*, *aua gronda*, *sbuvada*) as "debris flow". We therefore used two independent instruments. *Meaning:* back-translation to German with ZurichNLP's Romansh NLLB model, then the judge: guarded Romansh alerts preserve the meaning in **100 % (8B) / 97 % (70B)** (baseline 83 % / 93 %). *Vocabulary:* the official RG Hunspell dictionary flags non-standard words (Italian/French forms, other idioms, translated names) in most Romansh alerts; feeding the words back for repair raises dictionary-clean alerts only from 17–20 % to **27–33 %** (`docs/eval/alerts_rm_lexicon_guard.md`). Product consequence: the Romansh version is flagged to the commander with the suspect words; the German alert remains the backstop.

5.3 Chief-of-staff honesty. 6 game states × 12 questions (direct, derived, conditional, absent, pressure). Generic prompt: the model asserted an answer to an unanswerable question 20/28 (8B) and 19/28 (70B) times ("the bridge is no longer passable", "31 tourists"). Ours: **0/28** for both; overall 100 % (70B) / 93 % (8B) correct. The evaluation changed the product: v1 left subtraction to the model and the 8B over-refused; now all totals are computed in code, and caller-reported facts are rendered as labelled fields.

5.4 Press fact-check (unit tests). Numbers told to the journalist must exist in the situation picture *with the same meaning* ("120 people" is flagged even though "60 to 120 minutes" appears); casualty claims without a casualty report are flagged.

5.5 Cost and latency. Guarded alert in 4–5 languages: 5–9 s on CSCS, 65 s for 4 languages on the M1 laptop (8B). A complete scripted exercise (2 guarded alerts, 4 calls, 3 reports, subtitles, debrief) measured **51 calls, 16.7k input + 3.0k output tokens = CHF 0.019** at the published Swiss list price for Apertus 1.5 70B (Infomaniak: CHF 0.70 / 2.50 per 1M tokens; PHOENIQS: CHF 0.62 / 2.22). That is ~50 exercises per franc; on owned hardware the marginal cost is electricity.

5.6 Interaction design found by play-testing. The model misidentified an Italian reply as German, so language ID of staff replies is now deterministic (function words); callers' replies are generated *after* the game decides whether they understood, so dialogue and world never contradict; subtitles use the same terminology lock as alerts (an early subtitle rendered *Murgang* as "landslide").

6. Limitations

- The resident behaviour model is **illustrative, not empirically validated**: it encodes the mechanism (people act on the alert they can read), not real evacuation times. One scenario so far.
- Romansh vocabulary is not standard RG in most alerts, and no native speaker reviewed our Romansh; meaning was verified only by back-translation. Swiss German caller lines are a mix of dialect and Standard German.
- The semantic judge is itself a model; we treat it as secondary to the deterministic checks and report where it fails (Romansh).
- The deterministic hazard check knows five hazards and our listed synonyms; unseen correct synonyms count as "missing".
- Offline latency on a laptop is minutes, not seconds; real deployments would use a GPU server.
- Fictional names; the real-name substitution ("toponym snapping") is studied in our Track 1A companion submission.

7. Reproducibility

- `make test` runs 15 offline tests (deterministic world, alert reception by language, hazard classifier, check_text, press fact-check, Lagebild arithmetic, language ID, CAP export, AIRGAP guard, Romansh lexicon, no external assets,

session isolation).

- `make run` (from the repository root) builds and starts the game at http://localhost:8765 with `LLM_NAME`, `LLM_BASE_URL`, `LLM_API_KEY` from `.env` or the environment. Python 3.12 slim image; no GPU needed for the hosted mode.
- Air-gapped: `make llm-mac` + `make airgap-mac` (Mac, Metal) or `make airgap` (Docker, CPU/GPU) with the GGUF in `../../models`.
- Evaluation: `pip install -r requirements.txt -r requirements-eval.txt`; `cd src && python -m sturmglocke.evals alerts --n 30`, `... honesty --n 6`, `... rejudge --file <run>` (needs `JUDGE_*`), `... backtranslate --file <run>`; seeds fixed (cases 42, states 3, world 7); temperature 0 for all scored calls. Hardware used: MacBook Air M1 16 GB; CSCS for hosted inference.

8. Next steps

- Scenario editor and more hazards (flood below a dam, avalanche, rockfall) on real cantonal hazard maps (swisstopo OGD).
- A native-speaker Romansh review and a dedicated RG adapter (LoRA on Lia Rumantscha corpora), then re-measure.
- Pilot the Alert Desk with a cantonal civil-protection office; map our CAP 1.2 output to the CAP-Suisse profile once Switzerland's cell broadcast goes live.
- Multiplayer staff exercises (police, fire, civil protection, communication) and an instructor console.

License

Creative Commons Attribution 4.0 (CC-BY-4.0). All HackApertus projects are open-sourced. Code: Apache-2.0.

References

TERMDAT (Swiss Federal Chancellery terminology database) · Pledari Grond (Lia Rumantscha) · Lia Rumantscha RG Hunspell dictionary (MIT) · BABS / Alertswiss siren and warning pages · ZurichNLP romansh-nllb-1.3b · Apertus tech report arXiv:2509.14233 · Wikipedia: 2025 Blatten glacier collapse · BFS municipality count 1.1.2026 · privatim resolution on cloud services (24.11.2025).